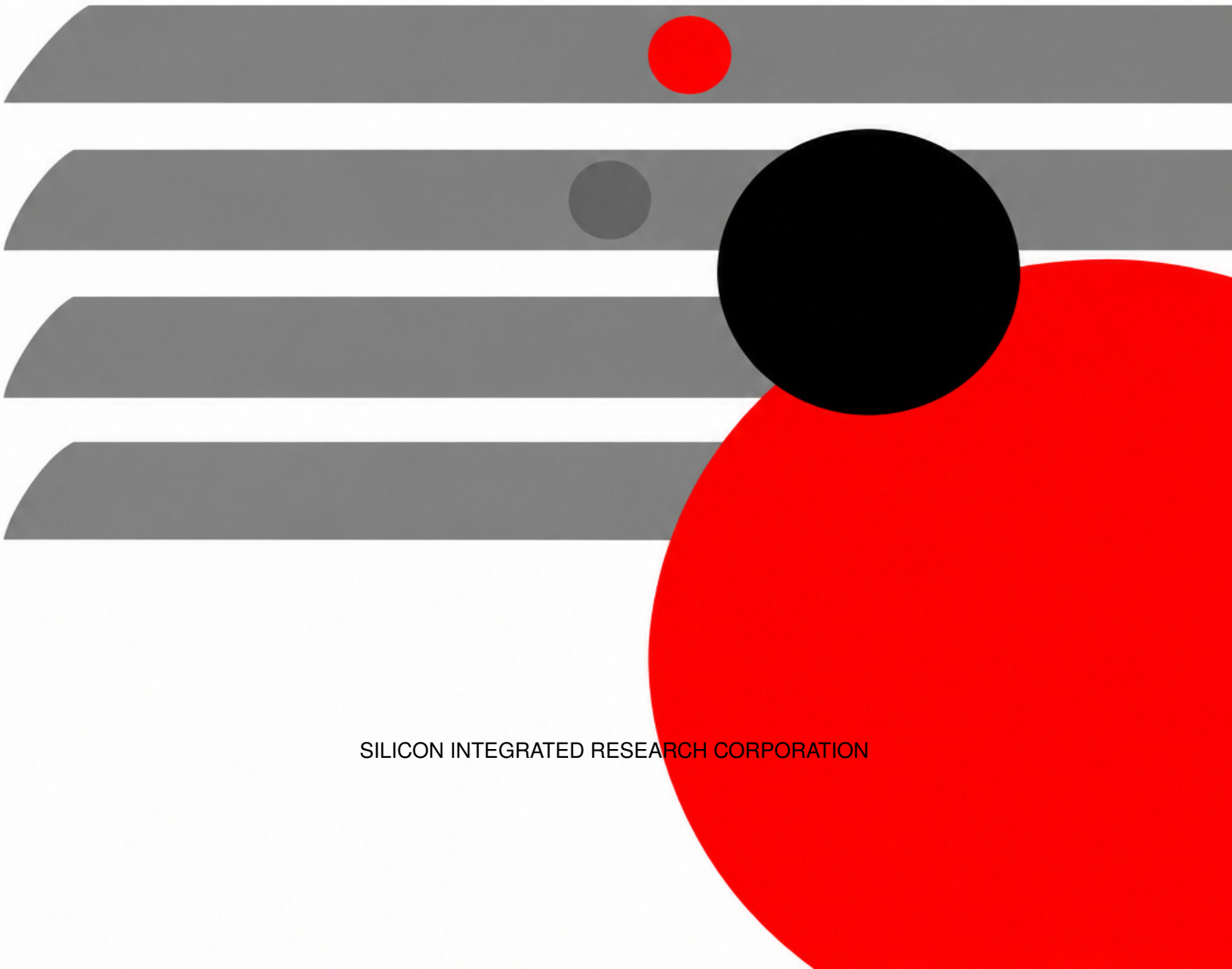


SIRC-1

Assembly Programmer's Manual

Assembly Language and Toolchain



SILICON INTEGRATED RESEARCH CORPORATION

Preface

About This Manual

This manual describes the SIRC assembly language and the tools that process it: the assembler, the linker, and the simulator. It is for programmers who write SIRC assembly. It is the companion to the *SIRC-1 Architecture Reference Manual*, which defines the instructions themselves, and the *SIRC-1 Hardware Reference Manual*, which describes the chip.

The *SIRC-1 Architecture Reference Manual* defines what each instruction does and gives the canonical notation used in its examples. This manual gives the complete language: how a source file is laid out, every operand form and directive, how symbols are resolved, and how to turn a source file into a program that runs.

Assumed Knowledge

This manual assumes the reader has read the Assembly Language Summary in the Notation and Glossary chapter of the *SIRC-1 Architecture Reference Manual* and is comfortable with the SIRC-1 register set and instruction formats.

Revision Status

This is Version 1.0 of the SIRC-1 Assembly Programmer's Manual, issued in draft for internal review and not for external distribution. It describes the behavior of the toolchain in this repository. Where the assembler does not yet implement something a programmer might expect, such as macros, Chapter 8 says so.

Contents

Preface iii

1 Introduction 1
- 1.1 Purpose and Scope . . . 1
- 1.2 The Toolchain . . . 1
- 1.3 Conventions . . . 1

2 Source Language 3
- 2.1 Lines, Whitespace, and Comments . . . 3
- 2.2 Case . . . 3
- 2.3 Names . . . 3
- 2.4 Numbers . . . 3
- 2.5 Registers . . . 4
- 2.6 Named Constants . . . 4

3 Operands 5
- 3.1 Operand Forms . . . 5
- 3.2 Immediate Operands . . . 5
- 3.3 Label References . . . 6
- 3.4 Shift Operands . . . 6
- 3.5 Register Shorthand . . . 6

4 Instruction Syntax 7
- 4.1 Anatomy of an Instruction . . . 7
- 4.2 Status Update Source . . . 7
- 4.3 Conditions . . . 7
- 4.4 Choosing the Immediate Format . . . 8
- 4.5 Mnemonics . . . 8

5 Directives and Symbols 9
- 5.1 Placing Code and Data . . . 9
- 5.2 Data Directives . . . 9
- 5.3 Named Constants . . . 9
- 5.4 Labels and Symbol References . . . 10

6 Using the Toolchain 11
- 6.1 A Worked Example . . . 11
- 6.2 The Assembler . . . 11
- 6.3 The Linker . . . 11
- 6.4 The Simulator . . . 12
- 6.5 Building with Make . . . 13

7 Diagnostics 15
- 7.1 Assembler Errors . . . 15
- 7.2 Linker Errors . . . 16

8 Limitations 17

List of Tables

1.1 The SIRC Manual Set 1
1.2 The SIRC Toolchain 1

2.1 Number Forms 3

3.1 Operand Forms 5

4.1 Condition Codes in Source 7
4.2 Mnemonics Recognized by the Assembler 8

5.1 Data Directives 9
5.2 Default Resolution of a Label Reference 10
5.3 Symbol Reference Suffixes 10

6.1 Segments Mapped by the Simulator 12

7.1 Common Assembler Diagnostics 16

8.1 Features the Assembler Does Not Have 17

List of Figures

CHAPTER 1

Introduction

1.1 Purpose and Scope

The SIRC-1 documentation is split into three manuals, each with its own scope:

Manual	Scope
SIRC-1 Architecture Reference Manual	ISA, instruction encodings, canonical assembly notation, exceptions, programmer-visible state
SIRC-1 Assembly Programmer's Manual	Complete assembly syntax, directives, symbols, meta-instructions, and toolchain usage (this manual)
SIRC-1 Hardware Reference Manual	Pinout, bus interface, timing, electrical characteristics

Table 1.1: The SIRC Manual Set

This manual covers the language and the toolchain. It does not define what an instruction does, which registers exist, or how exceptions work. For those, see the *SIRC-1 Architecture Reference Manual*; this manual refers to it by chapter name.

1.2 The Toolchain

A SIRC program is built in three steps, each with its own tool:

1. The **assembler** reads one assembly source file and writes an object file. It encodes every instruction and data directive, and records every label and every reference to a label.
2. The **linker** reads one or more object files, joins them end to end, resolves every reference to a label, and writes a flat binary program plus a debug map.
3. The **simulator** loads the binary at address zero, maps its fixed devices and any memory segments given on the command line, and runs it.

Tool	Reads	Writes
`assembler`	`.sasm` source	Object file (`.o`)
`linker`	One or more object files	Binary (`.bin`) and debug map (`.bin.dbg`)
`sirc_vm`	Binary	Register dump, terminal output

Table 1.2: The SIRC Toolchain

Chapter 6 describes how to run each tool.

1.3 Conventions

Source listings are shown in typewriter font. Mnemonics are bold typewriter, as in the *SIRC-1 Architecture Reference Manual*. File extensions are conventions, not requirements: the tools do not look at them.

CHAPTER 2

Source Language

2.1 Lines, Whitespace, and Comments

A source file is a sequence of instructions, directives and labels separated by whitespace. By convention each is written on its own line, with directives and instructions indented under the label they follow. Whitespace, including blank lines, is otherwise insignificant.

A comment starts with a semicolon and runs to the end of the line. It may follow an instruction on the same line.

```
1 ; a whole-line comment
2     MOVI r1, #1 ; a trailing comment
```

2.2 Case

Mnemonics, condition codes, shift types, and directives are written in uppercase. Register names are written in lowercase. The status update letters inside square brackets (Section 4.2) may be either case. Label and constant names are case-sensitive.

2.3 Names

Label names and constant names are made of letters, digits and underscores. A label is declared by writing a colon before its name, and referred to by writing an at sign before it:

```
1 :loop ; declare the label
2     BRAN @loop ; refer to it
```

The colon is written before the name, not after it. A label may be referred to before or after its declaration, and from any object file linked into the same program. A label name may be declared only once in a program.

2.4 Numbers

A number used as an operand value is prefixed with a hash sign. Three bases are accepted, and an underscore may be used anywhere inside the digits to make them easier to read.

Base	Written as	Examples
Decimal	`#` then digits, optional sign	`#42`, `#-3`, `#1_000`
Hexadecimal	`#0x` then hex digits	`#0x2A`, `#0xFF_00`
Binary	`#0b` then binary digits	`#0b0010_1010`

Table 2.1: Number Forms

A negative decimal number is stored in two's complement. The value must fit the field it is encoded into; for example, the immediate operand of an immediate-format instruction is 16 bits wide, and that of a short-immediate (shifted) instruction is 8 bits. The 8-bit short-immediate field and shift counts are unsigned, and the assembler rejects a negative number there.

Two directive operands are written without the hash. One is the address of `.ORG`, a bare decimal or `0x`-prefixed hexadecimal number; binary is not accepted (Section 5.1). The other is the name of `.EQU`, written `$NAME`.

2.5 Registers

The general-purpose registers are `r1` to `r7`. The status register is `sr`. Each address register pair is addressed as a whole by a single letter (`a`, `s`, `p`, or `l`) and as two separate halves: `ah` and `al`, `sh` and `sl`, `ph` and `pl`, `lh` and `ll`. The *SIRC-1 Architecture Reference Manual* describes what each register is for, and which are privileged.

2.6 Named Constants

`.EQU` defines a named constant. The name starts with a dollar sign at the definition and at every use:

```
1     .EQU $ANSWER #42
2
3     MOVI r1, $ANSWER ; no '#' at the use
```

A constant must be defined before its first use in the file. Defining the same name again changes its value for the lines that follow. A constant stands in for a numeric value in an immediate operand or in a data directive. It cannot be used as the address of `.ORG`.

CHAPTER 3

Operands

An instruction's operands are separated by commas, with the destination first. This chapter lists every operand form the assembler accepts. The entry for each instruction in the *SIRC-1 Architecture Reference Manual* gives the forms that instruction accepts, and the Addressing Modes chapter of the *SIRC-1 Architecture Reference Manual* gives the meaning of each addressing mode.

3.1 Operand Forms

Form	Name	Example
`#n`	Immediate number	`ADDI r1, #12`
`@label`	Immediate label reference	`MOVI al, @table`
`$NAME`	Immediate named constant	`CMPI r1, $LIMIT`
`r1`–`r7`, `sr`, halves	Register direct	`MOVR r2, r1`
`a`, `s`, `p`, `l`	Address register pair	`LJMP a`
`(addr)`	Indirect, zero displacement	`LOAD r1, (a)`
`(#n, addr)`	Indirect, immediate offset	`LOAD r1, (#4, a)`
`(r0, addr)`	Indirect, register offset	`STOR (r2, a), r1`
`(addr)+`	Post-increment, zero displacement	`LOAD r1, (a)+`
`(#n, addr)+`	Post-increment, immediate	`LOAD r1, (#0, a)+`
`(r0, addr)+`	Post-increment, register	`LOAD r1, (r2, s)+`
`-(addr)`	Pre-decrement, zero displacement	`STOR -(s), r1`
`-(#n, addr)`	Pre-decrement, immediate	`STOR -(#0, s), r1`
`-(r0, addr)`	Pre-decrement, register	`STOR -(r2, s), r1`
`type #count`, `type rC`	Shift	`ADDR r1, r2, ASL #1`

Table 3.1: Operand Forms

In the indirect forms `addr` is one of the four address register pairs, `a`, `s`, `p` or `l`. The displacement is added to the low word of the pair. The offset in parentheses may be a number, a label reference or a named constant.

An immediate displacement of zero may be omitted: `(addr)`, `(addr)+` and `-(addr)` are the same as `(#0, addr)`, `(#0, addr)+` and `-(#0, addr)`. The assembler accepts the shorthand only where it accepts the full form, and **`LJMP`** and **`LJSR`** take `addr` rather than `(addr)`. Register displacements have no shorthand.

3.2 Immediate Operands

An immediate operand may be a number, a label reference, or a named constant. A number is written with a leading hash (Table 2.1); a label reference and a named constant are not.

A label reference is accepted only by **`MOVI`**, **`LOAD`**, **`STOR`**, **`LDEA`**, **`LDEL`**, **`LJMP`**, **`LJSR`**, **`BRAN`** and **`BRSR`**. Other instructions with an immediate operand accept a number or a named constant only. To use an address with one of them, load it with **`MOVI`** first.

An immediate operand has the width of the instruction format it is encoded in: 16 bits in the immediate format and 8 bits in the short immediate format. The assembler is not required to diagnose a value that is too wide for its field, so a value must be kept in range.

3.3 Label References

A label reference in an operand is resolved by the linker, not the assembler. What it resolves to depends on the instruction and on an optional suffix; Chapter 5 gives the rules.

3.4 Shift Operands

A shift is written as a shift type followed by a count. The type is one of `NUL`, `LSL`, `LSR`, `ASL`, `ASR`, `RTL` or `RTR`. The count is either an immediate number from 0 to 15 or a register that holds the count. A space separates the type from the count.

```
1     ADDR r1, r2, LSL #2 ; literal count
2     ADDR r1, r2, LSR r3 ; count held in r3
```

A shift applies to one operand only, as defined for each instruction in the *SIRC-1 Architecture Reference Manual*. A count above 15 is rejected by the assembler. The Shift Operations chapter of the *SIRC-1 Architecture Reference Manual* defines each shift type.

3.5 Register Shorthand

A save-result register-form ALU instruction, such as **ADDR**, written with two register operands is shorthand for the three-operand form with the destination repeated as the first source. **MOVR** is the exception: its only form is `MOVR rD, rS`. These two lines assemble to the same instruction:

```
1     ADDR r1, r1, r2 ; r1 = r1 + r2
2     ADDR r1, r2 ; r1 = r1 + r2
```

A shift written with the shorthand applies to the left operand, which is the destination register, not to the second register written.

The test instructions **CMNR**, **CNCR**, **CMPR**, **CPCR**, **TSAR**, **TSOR** and **TSXR** have no destination. Each takes exactly two registers, `rS1, rS2{, shift}`.

CHAPTER 4

Instruction Syntax

4.1 Anatomy of an Instruction

An instruction is written as a mnemonic, an optional status update source (`<SUS>`), an optional condition (`<COND>`), and then its operands:

```
<MNEMONIC>{[<SUS>]}{|<COND>} <OPERAND>, ...
```

The three parts must be written in that order and with no spaces between them. The operands follow after at least one space.

```
1    ADDI r1, #1 ; mnemonic and operands
2    ADDI[N] r1, #1 ; with a status update source
3    ADDI|== r1, #1 ; with a condition
4    ADDI[N]|== r1, #1 ; with both
```

4.2 Status Update Source

`[N]`, `[A]` or `[S]` after the mnemonic selects what updates the status flags: nothing, the ALU result, or the shifter result. Without it the instruction uses its default, which is `[A]` for most ALU instructions. The default of each instruction and the meaning of each source are in the *SIRC-1 Architecture Reference Manual*.

Only ALU instructions have a status update source. **`MOVI`** and **`MOVR`** accept `[A]` and `[N]` but not `[S]`, because they have no shifted operand. Memory and control-flow instructions never update the flags and must not be written with a status update source.

4.3 Conditions

`|<COND>` after the mnemonic, and after the status update source if there is one, makes the instruction conditional. The sixteen conditions are:

Written as	Meaning
`AL`	Always (the default when no condition is written)
`==`, `!=`	Equal, not equal
`CS`, `CC`	Carry set, carry clear
`NS`, `NC`	Negative set, negative clear
`OS`, `OC`	Overflow set, overflow clear
`HI`, `LO`	Unsigned higher, unsigned lower or same
`>=`, «, », `<=`	Signed comparisons
`NV`	Never

Table 4.1: Condition Codes in Source

The Condition Codes chapter of the *SIRC-1 Architecture Reference Manual* defines the flags each condition tests and how to choose between them.

4.4 Choosing the Immediate Format

The assembler picks the encoding from how the operands are written. For the ALU instructions that have both forms, an immediate operand on its own is encoded in the 16-bit immediate format, and an immediate operand followed by a shift is encoded in the 8-bit short immediate format:

```
1     ADDI r1, #5 ; immediate format, 16-bit value
2     ADDI r1, #5, LSL #1 ; short immediate format, 8-bit value
```

The register format is selected by the mnemonic: write **ADDR** with register operands, not **ADDI**.

4.5 Mnemonics

The assembler recognizes the mnemonics in Table 4.2. Each is documented in the chapter of the *SIRC-1 Architecture Reference Manual* named in the table.

Family	Mnemonics	*SIRC-1 Architecture Reference Manual* Chapter
ALU	**ADDI**/**ADDR**, **ADCI**/**ADCR**, **SUBI**/**SUBR**, **SBCI**/**SBCR**, **ANDI**/**ANDR**, **ORRI**/**ORRR**, **XORI**/**XORR**, **MOVI**/**MOVR**	ALU Instructions
ALU test	**CMNI**/**CMNR**, **CNCI**/**CNCR**, **CMPI**/**CMPR**, **CPCI**/**CPCR**, **TSAI**/**TSAR**, **TSOI**/**TSOR**, **TSXI**/**TSXR**	ALU Instructions
Memory	**LOAD**, **STOR**	Memory Access Instructions
Addresses	**LDEA**, **LDEL**, **LJMP**, **LJSR**	Control Flow Instructions
Branches and returns	**BRAN**, **BRSR**, **RETS**	Control Flow Instructions
Coprocessor	**COPI**, **COPR**, **EXCP**, **WAIT**, **RETE**, **RSET**, **ETFR**, **ETTR**	Coprocessor Instructions
Maths and DMA	**MULU**, **MULS**, **DIVU**, **DIVS**, **DMAR**, **DMAW**, **DMAT**	Coprocessor Instructions
Other meta	**NOOP**, **SHFT**	Meta-Instructions, ALU Instructions

Table 4.2: Mnemonics Recognized by the Assembler

Many of these are *meta-instructions*: names for a single real instruction encoding. The Meta-Instructions chapter of the *SIRC-1 Architecture Reference Manual* lists what each one assembles to. The assembler has no pseudoinstructions that expand to more than one instruction.

CHAPTER 5

Directives and Symbols

5.1 Placing Code and Data

`.ORG addr` sets the word address at which the following instructions and data are assembled. The address is a bare decimal or `0x`-prefixed hexadecimal number, without the hash used for immediate operands. It must be even, because instructions and data directives each occupy two words. `.ORG` may appear any number of times, to put code or data at fixed addresses such as the reset and exception vector table.

In a program linked from several objects, `.ORG` is relative to the start of its own object, and each object starts where the previous one ends. The vector table and any other fixed-address code must therefore go in the first object.

```
1      .ORG 0x0000
2      .DQ @start ; reset vector
3
4      .ORG 0x0200
5  :start
6      NOOP
```

The vector table and the address of each vector are defined in the Exception and Interrupt Handling chapter of the *SIRC-1 Architecture Reference Manual*. Addresses are word addresses, not byte addresses.

5.2 Data Directives

`.DB`, `.DW` and `.DQ` each reserve one slot of two words (4 bytes), the same size as an instruction, and fill it with a value. Despite their names they differ only in where the value is placed within the slot:

Directive	Value goes in	Example result
`.DB #0xAB`	Low byte	Bytes `00 00 00 AB`
`.DW #0xABCD`	Low word	Bytes `00 00 AB CD`
`.DQ #0xABCD1234`	Whole slot	Bytes `AB CD 12 34`

Table 5.1: Data Directives

`.DB` does not pack four bytes into a slot, and `.DW` does not pack two words. The directive should be chosen to document the intended width of the value. Each may also take a named constant, which is placed and truncated like a number, or a label reference. A label reference with no suffix always fills the whole slot with the label's address, whichever of the three directives is used. With `.l`, `.u` or `.r`, the 16-bit value goes in the low word.

5.3 Named Constants

`.EQU $NAME #value` defines a constant. The constant is referred to as `$NAME`, without a hash, wherever the number would be used. A constant may be used in an immediate operand and in a data directive.

```
1     .EQU $UART_RECV #0x0001
2
3     MOVI al, $UART_RECV
```

5.4 Labels and Symbol References

A label is declared with a leading colon and referred to with a leading at sign. The assembler records the references; the linker resolves them once it knows where every label ended up, so a reference may be to a label that appears later in the file or in a different object file.

What a reference resolves to depends on where it is written:

Where	Default resolution
BRAN, **BRSR**	16-bit signed word displacement f instruction to the label
Other instruction operands (**MOVI**, **LOAD**, **STOR**, **LDEA**, **LDEL**, **LJMP**, **LJSR**)	The label's low word only
`.DB`, `.DW`, `.DQ`	The label's full 24-bit address

Table 5.2: Default Resolution of a Label Reference

A suffix overrides the default:

Written as	Resolves to
`@label.r`	The PC-relative word displacement
`@label.l`	The low word of the address
`@label.u`	The high word (segment) of the address

Table 5.3: Symbol Reference Suffixes

A full 24-bit address is loaded into an address register pair in two halves, as follows:

```
1     MOVI ah, @table.u ; segment (high word)
2     MOVI al, @table.l ; offset (low word)
```

A displacement that does not fit in 16 signed bits is a link-time error, and so is a reference to a label that is not declared in any linked object file (Chapter 7).

CHAPTER 6

Using the Toolchain

This chapter walks through building and running a program with the three tools. In this repository they are built from the `sirc-vm` Cargo workspace. The commands below show the tool names. Putting `cargo run -bin tool --` in front of the arguments builds and runs a tool in one step.

6.1 A Worked Example

Save this program as `count.sasm`. It counts `r1` up to ten and stops the simulator.

```
    .ORG 0x0000
    .DQ @start ; reset vector

    .ORG 0x0200
:start
    MOVI r1, #0
:loop
    ADDI r1, #1
    CMPI r1, #10
    BRAN|!= @loop
    COPI #0x14FF ; stop the simulator (test exit)
```

Assemble, link, and run it:

```
assembler --input-file count.sasm --output-file count.o
linker --segment-offset 0 --output-file count.bin count.o
sirc_vm --program-file count.bin --register-dump-file count.register-dump
```

The register dump written at the end shows `r1` holding `0xA`.

The final **`COPI`** with `#0x14FF` is a test hook that only the simulator implements. It stops the simulator. On a conforming CPU, the instruction raises an invalid-opcode fault, because exception coprocessor opcode `0x4` is reserved (see the Exception and Interrupt Handling chapter of the *SIRC-1 Architecture Reference Manual*). Programs for real hardware must not use it.

6.2 The Assembler

```
assembler --input-file <FILE> --output-file <FILE>
```

The assembler reads one source file and writes one object file. Both options are required and have the short forms `-i` and `-o`. The assembler stops at the first syntax error and prints the offending line (Chapter 7).

The object file is a binary file for the linker. It holds the assembled program, the labels the file declares, the references it makes, and a copy of the source for the debug map. It is not meant to be read or edited.

6.3 The Linker

```
linker --segment-offset <N> --output-file <FILE> <OBJECT FILE> ...
```

The linker joins its input objects in the order they are given, so the first object starts at the lowest address. A label declared in any object can be referred to from any other. The linker then patches every reference using the final address of its label, and writes:

- the program as a flat binary at the output path, and
- a debug map at the output path with `.dbg` added, which relates program addresses to source lines.

`--segment-offset` (short form `-s`) is added to every label's address before it is patched in. The option is required. `N` is a decimal number of words; a hexadecimal value such as `0x10` is rejected. This differs from the simulator's `--segment` offsets, which are hexadecimal. A program that is mapped into its own memory segment, as every example in the *SIRC-1 Architecture Reference Manual* is, is linked with an offset of 0.

A program made of several files is linked in one step:

```
linker --segment-offset 0 --output-file prog.bin main.o data.o
```

6.4 The Simulator

```
sirc_vm --program-file <FILE> {--segment <SEGMENT>} ... {--register-dump-
    file <FILE>}
```

The simulator loads the binary at the start of its program memory and starts the CPU through the reset vector.

`--program-file`, `-p` The linked binary to load.

`--segment`, `-s` Maps additional memory, written `label:offset:length` or `label:offset:length:file`, with the offset and length in hexadecimal. With a file, the segment is mapped onto it: the program reads the file's contents, and every store to the segment writes through to the file. The file must exist and be writable. This option may be repeated.

`--register-dump-file`, `-r` Writes the final register state to a file when the program stops, which is how the repository's examples check their results.

`--enable-video`, `-e` Maps a video device in the VIDEO segment (Table 6.1). The option is available only when the simulator is built with the `video` feature.

`-v`, `-q` Raise or lower the amount of logging. `-vv` is common while developing.

`--debug`, `-d` Starts a debug adapter for an editor to attach to, using the debug map written by the linker.

The simulator always maps the segments in Table 6.1. Offsets and lengths are word addresses in hexadecimal. A `--segment` must not overlap them.

Segment	Offset	Length	Notes
PROGRAM	`0x000000`	`0xFFFF`	Holds the linked binary; read-only
TERMINAL	`0x0A0000`	`0xF`	Terminal device
DEBUG	`0x0B0000`	`0xF`	Debug device
VIDEO	`0x0C0000`	`0xFFFF`	Mapped only with `--enable-video`

Table 6.1: Segments Mapped by the Simulator

For example, a program that needs 64 K words of scratch memory away from the program binary is run with:

```
sirc_vm --program-file prog.bin --segment SCRATCH:00010000:10000
```

6.5 Building with Make

A `Makefile` keeps the three steps in sync. This is the pattern used by the examples in the repository:

```
prog.o: prog.sasm
	assembler --input-file prog.sasm --output-file prog.o

prog.bin: prog.o
	linker --segment-offset 0 --output-file prog.bin prog.o

run: prog.bin
	sirc_vm -vv --program-file prog.bin
```

CHAPTER 7

Diagnostics

7.1 Assembler Errors

The assembler stops at the first error. It prints the file, line and column, the lines around the error with a marker under the offending token, and the chain of syntax contexts it was inside. The final line states the reason:

```
error: parsing failed
  --> bad.sasm:6:10
4 | ADDI r1, #5
5 | ADDI r1, #5, LSL #1
6 | LOAD r1, #5
  | ^

in section "instruction" at line 6, column 5,
in section "LOAD instruction" at line 6, column 5,
external error:
  Invalid addressing mode for LOAD: LOAD only reads memory. Use MOVI to
     load an
  immediate value into a register. at line 6, column 10
```

Where the instruction has a correct alternative, the diagnostic names it. The assembler appends `at line` *n*`, column` *m* to each message. The diagnostics for the most common mistakes are:

Source	What the Assembler Says	Meaning
`LOAD r1, #5`	`Invalid addressing mode for LOAD: LOAD only reads memory. Use MOVI to load an immediate value into a register.`	**LOAD** only reads memory; use **MOVI**
`LOAD r1, r2`	`Invalid addressing mode for LOAD: LOAD only reads memory. Use MOVR to copy one register to another.`	**LOAD** only reads memory; use **MOVR**
`LOAD r1, (r2, a), LSL #1`	`Invalid addressing mode for LOAD: LOAD does not support a shift. Use SHFT on the offset register before the LOAD, or on the destination register after it.`	**LOAD** has no shift; use **SHFT**
`MOVI[S] r1, #5`	`The [MOVI] opcode does not support the [S] status register update source because it has no shift operand. Use [A] or [N].`	`[S]` needs a shift operand; use `[A]` or `[N]`
`MOVR r1, r2, LSL #1`	`Invalid addressing mode for MOVR:. Cannot use a shift with direct register -> direct register moves. Use SHFT instruction instead.`	A register-to-register move has no shift; use **SHFT**
`ADDR r1, r2, LSL #16`	`Shift definitions can only be in the range of 0-15, got 16`	The shift count must be 0–15
`LOAD ah, (#0, a)+`	`Aliased register writes are undefined for LOAD: destination register overlaps the post-increment address register`	The destination is half of the updated address register

Table 7.1: Common Assembler Diagnostics

The `ADDR` diagnostic is wrapped in the text `Parsing Failure: Base { location: ..., kind: External("...") }`; the table shows the inner message. The aliased-write diagnostic ends with a different phrase for each form. It is reported for a pre-decrement or post-increment form whose other register is a half of the updated address register, and for an **LDEL** or **LJSR** that would write `l` twice. The Memory Access Instructions and Control Flow Instructions chapters of the *SIRC-1 Architecture Reference Manual* give the exact list.

When an instruction is given operands it does not accept, the diagnostic names the instruction and either shows the operands it parsed or gives an example of an accepted form. When an operand cannot be parsed at all, the diagnostic lists the tokens the assembler expected at that point.

7.2 Linker Errors

The linker reports two problems, each as a failure with the cause in the message:

- **Unresolved symbol.** `Cannot find symbol [name] in symbol definitions.` A reference names a label that no linked object declares. Check the spelling, and that the object file that declares it is on the linker's command line.
- **Displacement out of range.** `Offset ... does not fit into a 16 bit signed integer.` A **BRAN** or **BRSR** (or a reference forced with `.r`) is further from its label than a 16-bit signed word displacement can reach. Use **LJMP** or **LJSR** for a long jump.

CHAPTER 8

Limitations

The assembler in this repository is deliberately small. The following are *not* supported. A listing that depends on any of them will not assemble.

Feature	Instead
Macros	Repeat the instructions, or generate the source
File inclusion	Assemble each file separately and link the objects t
Expressions and arithmetic on labels	Compute the value by hand and write a number, or
Conditional assembly	Maintain separate source files
Named sections or segments	Place code and data with `.ORG`
Local labels	Give every label a unique name
Listing and map files	Use the debug map written by the linker
Multi-instruction pseudoinstructions	Write the instructions explicitly
Label references in ALU and coprocessor immediates	Load the address with **`MOVI`** first

Table 8.1: Features the Assembler Does Not Have

The architecture requires that every meta-instruction assemble to exactly one instruction, so every mnemonic written in the source produces exactly one instruction (see the Meta-Instructions chapter of the *SIRC-1 Architecture Reference Manual*).

Four behaviors are worth knowing about:

- The assembler stops at the first error, so a file with several mistakes needs several runs.
- The assembler does not promise to reject an immediate value that is wider than its field.
- The assembler masks the link-register index of **`ETFR`** and **`ETTR`** to four bits without a diagnostic. The architecture leaves values above `#7` architecturally undefined (see the Coprocessor Instructions chapter of the *SIRC-1 Architecture Reference Manual*).
- The **`COPI`** instruction with `#0x14FF` is a simulator-only test hook. On a conforming CPU it raises an invalid-opcode fault.

This chapter will shrink as the toolchain grows. A feature listed here is not an oversight in the manual.

SIRC-1 Assembly Programmer's Manual

This manual is the working guide to writing SIRC-1 assembly: the source language, every operand form and directive, how labels and constants are resolved, and how to assemble, link and run a program with the toolchain. It leaves the definition of each instruction to the companion *SIRC-1 Architecture Reference Manual*.

This manual contains:

- The lexical rules and operand syntax of the language
- Condition codes, status update sources and shifts as they are written in source
- Directives, labels, constants and symbol references
- Step-by-step use of the assembler, linker and simulator
- The diagnostics the assembler produces and what the toolchain does not support

SILICON INTEGRATED RESEARCH CORPORATION
1400 Fabrication Way, Milpitas Flat, CA
Printed in the United States of America

First Printing, 1989 • Version 1.0